IEEE Access
Multidisciplinary : Rapid Review : Open Access Journal

Received 31 October 2025, accepted 19 November 2025, date of publication 26 November 2025, date of current version 8 December 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3637505

RESEARCH ARTICLE

SecLLM: Enhancing Security Smell Detection in IaC With Large Language Models

GABRIELE DE VITO, (Member, IEEE), FABIO PALOMBA, (Member, IEEE), AND FILOMENA FERRUCCI
Department of Computer Science, University of Salerno, 84084 Fisciano, Italy
Corresponding author: Gabriele De Vito (gadevito@unisa.it)

This work was supported in part by the Italian Piano Nazionale di Ripresa e Resilienza (PNRR) Ministero dell'Università e della Ricerca (MUR) under Project PE0000013-FAIR.

ABSTRACT The emergence and expansion of Infrastructure-as-Code (IaC) paradigms have transformed cloud provisioning methodologies, while simultaneously introducing complex security smells. Traditional detectors are unable to handle the semantic challenges (i.e., understanding the meaning and context of code beyond its syntax) introduced by contemporary IaC ecosystems due to their reliance on heuristic rules and syntactic patterns. This work introduces SecLLM, an innovative framework that leverages the capabilities of Large Language Models (LLMs) to reconceptualize the detection of security smells through contextual semantic analysis. Our approach avoids conventional intermediate representations, exploiting natural language understanding to examine Ansible, Puppet, and Chef configurations through adaptive prompt engineering methodologies. The framework architecture supports multiple IaC infrastructures and LLMs while simultaneously enabling optimization of the underlying LLM responses for each of the nine security smell analyzed. Furthermore, SecLLM provides detailed feedback regarding the analyzed scripts that can help practitioners define their infrastructure's risk profile and resolve the identified smells. The empirical validation demonstrates that SecLLM outperforms GLITCH, a state-of-the-art security smell identification tool, achieving precision improvements of 12-21 percentage points and F1-score gains of 17-32 percentage points in terms of detection accuracy while simultaneously maintaining low operational costs ($0.003-$0.015 per script). Additionally, we address the non-deterministic behavior inherent to LLMs through three mitigation strategies (confidence-based filtering, ensemble voting, and response caching) that achieve perfect reproducibility (Fleiss' k = 1.00) and preserve detection accuracy.

INDEX TERMS Devops, infrastructure as code, security smells, large language models.

I. INTRODUCTION

Cloud computing has completely changed how modern organizations manage their IT infrastructure, requiring scalable and repeatable approaches to handle increasingly complex deployments. In response to this challenge, Infrastructure-as-Code (IaC) has enabled practitioners to define and provision infrastructure through machine-readable configuration files, replacing error-prone manual processes. As a result, IaC tools such as Ansible,[1] Puppet,[2] and Chef[3] have become integral to DevOps practices [1], [2], [3], [4].

Despite the advantages of these tools, IaC scripts themselves may contain suboptimal patterns or configurations, called security smells, which, if not properly managed, can compromise the integrity and security of the entire infrastructure [1], [5], [6], [7].

However, the diversity of IaC technologies, with their own syntax and semantics, makes the application of a universally

The associate editor coordinating the review of this manuscript and approving it for publication was Byung-Seo Kim.

[1]https://www.ansible.com/
[2]https://www.puppet.com/
[3]https://www.chef.io

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

valid approach for security smells detection challenging [7], [8].

Current methods often rely on intermediate representations for technology-agnostic analysis or on predefined rules and patterns [5], [8], [9], [10]. A significant limitation of these approaches is the potential loss of context and language-specific details of each IaC language, which can reduce the effectiveness of the detection itself.

To better illustrate these limitations, consider the Puppet script in Figure 1, where hard-coded credentials are embedded within nested data structures rather than direct assignments. The passwords 'supersecret' are accessible through complex navigation such as '$db_configs['credentials'][1]' and '$db_configs['connection']['users']['secret']'. Static rule-based detectors that search for direct string assignments may fail to identify these obfuscated secrets, as a semantic understanding of data flow and structures is required.

```
1  $db_configs = {
2      'credentials' => ['root', 'supersecret'],
3      'connection' => {
4          'host' => 'myhost.com',
5          'users' => {'admin' => 'root', 'secret' => 'supersecret'}
6      }
7  }
8
9  define mysql::datasource($rootpw, $ds_name, $ds_owner, $ds_owner_pwd,
10                          $ds_user, $ds_user_pwd, $ds_schema) {
11     $mysql_root_cmd = "/usr/bin/mysql -u ${db_configs['credentials'] [0]}"+
12                       "-p${db_configs['credentials'] [1]}"
13
14     exec { "create datasource ${ds_name}":
15         command => "/usr/bin/mysqladmin -u ${db_configs['connection'] ['users'] ['admin']}"+
16                    "-p${db_configs['connection'] ['users'] ['secret']} create ${ds_name}",
17     }
18 }
```

FIGURE 1. Hard-coded secret example in puppet.

In the Ansible script illustrated in Figure 2, the default admin user is set through a variable with a fallback value, which can be missed by static analysis tools that do not evaluate variable defaults or conditional logic.

```
1    - name: Configure database
2      vars:
3        db_user: "{{ default_user | default('admin') }}"
4      command: mysqladmin -u "{{ db_user }}" password 'newpassword'
```

FIGURE 2. Admin by default example in ansible.

These examples underscore the need for detection techniques that can grasp the semantics and context of IaC scripts. Moreover, traditional tools struggle to keep pace with emerging security smells and evolving IaC technologies, limiting their effectiveness in dynamic environments where rapid adaptation to new threats is essential.

Large Language Models (LLMs) have emerged as powerful tools in the field of natural language processing, thanks to their ability to understand and generate human-like text [11], [12], [13].

Given the impressive performance of these models, their potential in the context of IaC security analysis is worth exploring.

However, the use of LLMs also presents challenges, such as non-determinism in their outputs, which can affect the consistency and reliability of security smell detection [14], [15]. This non-determinism arises from the probabilistic nature of the strategies used by LLMs to generate responses, which can produce different results for the same input.

To verify whether LLMs can be effective in this domain, we propose a framework, named SecLLM. SecLLM supports nine types of smells across Puppet, Ansible, and Chef, and operates through a prompt-based analysis pipeline that processes IaC scripts in their native format, without intermediate representations. The flexibility to use different LLMs for different security smells is a unique feature of the proposed framework. This feature enhances detection accuracy by tailoring the analysis to the specific requirements and characteristics of each security smell.

SecLLM provides comprehensive feedback for each analysis, helping developers validate their security practices and safely address code smells.

We conducted a comparative evaluation of SecLLM against GLITCH [8], a state-of-the-art security smell detector, using different LLMs (GPT-4o-mini, Qwen-2.5 14B and 32B). The results reveal that our method simplifies the process of security smell analysis across various IaC technologies and surpasses the baseline in terms of precision and recall at affordable costs. To ensure practical deployability, we also evaluated LLM non-determinism and implemented three mitigation strategies (response caching, ensemble voting, and confidence-based filtering), achieving perfect reproducibility (Fleiss' Kappa = 1.00) while maintaining detection accuracy.

The key contributions of this paper are as follows:

1) The introduction of a novel framework that utilizes LLMs for direct analysis of IaC scripts, eliminating the need for intermediate representations and predetermined rules. It offers flexibility with different LLMs for specific security smells and customizable prompts for Ansible, Puppet, and Chef.
2) An empirical evaluation comparing SecLLM against GLITCH, analyzing computational costs, token consumption patterns, and the non-deterministic behavior of LLMs, was conducted to assess consistency and reliability in security smell detection.
3) A replication package that includes all the datasets, scripts, and configurations used in our experiments. This package aims to support the reproducibility of our results and encourage further exploration in the field of IaC security analysis.

A. STRUCTURE OF THE PAPER

The remainder of the paper is organized as follows. **Section** II provides background and covers the related work. **Section** III presents the SecLLM approach, while the research method employed to assess it is described in **Section** IV. **Section** V reports and discusses the results, and analyzes the threats to the validity. **Section** VII concludes the paper and provides future work.

II. BACKGROUND AND RELATED WORK

In this section, we overview the necessary background to comprehend our work and its relevant context. First, we offer

TABLE 1. Security smells taxonomy for IaC scripts showing nine smell types across three categories (Credential, Network, Code Quality) with corresponding CWE mappings and vulnerability descriptions.

Category	Security Smell	CWE	Description
Credential	Admin by Default	250	Default users assigned administrative privileges without explicit configuration, violating least privilege principle
	Empty Password	258	Password attributes set to empty strings, significantly weakening authentication mechanisms
	Hard-coded Secret	798/259	Sensitive credentials embedded directly in scripts rather than using secure credential management
Network	Unrestricted IP Address	284	Services bound to 0.0.0.0, allowing connections from any network and exposing them to unauthorized access
	Use of HTTP without TLS	319	Unencrypted HTTP communication making data susceptible to interception and MITM attacks
Code Quality	No Integrity Check	353	Downloaded content lacks integrity verification through checksums or cryptographic signatures
	Use of Weak Cryptography	327/326	Outdated cryptographic algorithms (MD5, SHA-1) vulnerable to various attacks
	Suspicious Comment	546	Comments indicating defects, missing functionality, or weaknesses (TODO, FIXME) revealing areas requiring attention
	Missing Default Case	478	Conditional statements lacking default cases, potentially leaving input combinations unhandled

a brief introduction to IaC scripts and the concept of security smells in IaC. Then, we delve into LLMs, which form the foundation of the proposed framework. Finally, we review the related work.

A. IaC SCRIPTS

IaC scripts consist of declarative or imperative commands designed to automate the provisioning, configuration, and management of infrastructure and associated services [4]. Ansible, Chef, and Puppet are prominent open-source automation tools that support IaC, each offering unique functionalities and approaches to meet diverse needs within the DevOps community.

Ansible uses an agentless architecture with YAML-based playbooks and communicates via SSH to external environments without requiring software installation on target machines [16], [17]. Because of its modular architecture, roles—i.e., groups of tasks, variables, and templates—can be reused, improving maintainability across complex infrastructures.

Chef employs a client-server architecture with scripts, namely cookbooks, written in Ruby. The Chef server acts as a central repository for configuration data, while clients installed on target machines apply configurations. Chef excels in flexibility and extensibility, supporting roles and environments to organize configurations across deployment stages. Roles apply settings to node groups, while environments separate configurations for development, testing, and production.

Puppet operates on a client-server model using a declarative language in manifests. Its resource abstraction layer allows platform-independent configuration management, enabling the same manifest to be used across different operating systems. The declarative approach specifies desired resource states while automatically determining the implementation steps. Despite architectural differences, all three tools share security-critical characteristics that enable security issues or misconfiguration propagation: they handle privileged credentials, use dynamically-typed configuration languages without compile-time safety checks, and employ abstraction layers that obscure the security impact of directives (e.g., binding to 0.0.0.0 or downloading artifacts without integrity verification). These commonalities motivate technology-agnostic detection solutions.

B. SECURITY SMELLS IN IaC

The security-critical characteristics of IaC tools described above manifest as suboptimal coding practices that can lead to potential vulnerabilities or security risks, called security smells [5]. These smells indicate patterns that, while not necessarily vulnerabilities themselves, could facilitate security breaches if left unaddressed [5]. Rahman et al. have made significant contributions in this area by identifying nine security smells and linking them to the Common Weakness Enumeration (CWE) taxonomy [18], providing a standardized framework for understanding these issues [5], [19], [20].

Table 1 summarizes the nine security smells addressed in this work, which span three risk categories: credential management, communication security, and code quality indicators. For detailed definitions and examples, we refer readers to the original studies [5], [19], [20].

However, detecting these smells in practice is non-trivial. As Figures 1 and 2 illustrate, security smells often hide behind obfuscated variable references, nested data structures, and conditional defaults—patterns that require understanding code semantics and execution context beyond what syntactic pattern matching can provide.

C. LARGE LANGUAGE MODELS

LLMs are neural networks with extensive parameters trained on vast text corpora, offering promising capabilities in generating human-like text [21], [22], [23] and semantic understanding, requirements necessary in the context of security smells.

Recent applications in software engineering have demonstrated the capability of LLMs to analyze code semantics, identify anti-patterns, and provide contextual explanations [12], [13]. Specifically in the IaC domain, Hassan et al. [24] showed that LLMs can provide analyses that traditional

static analysis tools are unable to offer, particularly for patterns requiring semantic disambiguation—such as distinguishing between legitimate variable indirection and obfuscated credential storage. Today, there is a wide array of LLMs available, ranging from commercial options like GPT-4 [25] and CLAUDE 3.5 [26] to open-source alternatives like LLama [27], each offering unique capabilities and limitations.

However, the deployment of LLMs in security-critical applications presents challenges, notably non-determinism in outputs due to their probabilistic generation strategies [14], [15]. This variability can affect consistency and reliability in detection results, requiring careful framework design to ensure reproducible and trustworthy security assessments across different executions.

D. RELATED WORK

Several studies have focused on code quality and secure coding practices within IaC. Sharma et al. [28] conducted a study on IaC quality, introducing a catalog of 24 configuration smells (13 implementations and 11 design smells) that breach best practices. Analyzing 4,621 Puppet repositories with over 8.9 million lines of code, they find design smells co-occur 9% more than implementation smells, linking poor design choices to multiple issues. The authors created Puppeteer to detect design smells and use Puppet-Lint for implementation smells, emphasizing the need for better practices and tools to enhance configuration code quality for researchers and practitioners.

Schwarz et al. [29] extended the work of Sharma et al. by adapting Puppet IaC smells to Chef and propose a new set of Chef-specific IaC smells. They classified IaC smells into three categories: technology agnostic, technology dependent, and technology specific. They introduced a detection tool using Foodcritic for Chef cookbooks, evaluated on industry and Chef supermarket datasets. Findings show many IaC smells are common across technologies, though some are more context-specific.

More relevant to our work are the studies from Rahman et al. [5], [9], [10], Saavedra et al. [8], and Opdebeeck et al. [30]. Rahman et al. [5] conducted an empirical study on security smells in IaC scripts, which indicate potential security weaknesses. Analyzing 1,726 scripts, they identified seven security smells, such as hard-coded secrets and weak cryptography. They developed SLIC, a static analysis tool, to detect these smells, achieving a precision and recall of 0.99 using an oracle dataset. However, Reis et al. [31] replicated SLIC's evaluation and found that its precision is substantially lower than originally reported after validation with code owners, dropping from 99% to 28%.

In [9] Rahman et al. developed a static analysis tool named Security Linter for Ansible and Chef scripts (SLAC) to automatically detect security smells. SLAC was evaluated using oracle datasets, achieving high precision and recall, indicating its reliability in identifying security smells.

TABLE 2. Summary of related work in IaC smell detection. Comparison of existing tools by supported IaC technologies, detected smells, and detection approaches.

Study	IaC Script	Security Smells	Identification Approach
Puppeteer [28]	Puppet	Missing default case Suspicious Comment	Metric-Based Static Analysis
Foodcritic [29]	Chef	None	Static Analysis
SLIC [5]	Puppet	Admin by default Empty password Hard-coded secret Invalid IP binding Suspicious Comment HTTP without TLS	Rule-Based Static Analysis
SLAC [9]	Ansible Chef	Admin by default Empty password Hard-coded secret Invalid IP binding Suspicious Comment HTTP without TLS Missing default case No integrity check Weak crypto alg.	Rule-Based Static Analysis
GLITCH [8]	Ansible Chef Puppet	Admin by default Empty password Hard-coded secret Invalid IP binding Suspicious Comment HTTP without TLS Missing default case No integrity check Weak crypto alg.	Intermediate Representation and Rule Matching
GASEL [30]	Ansible	Admin by default Empty password Hard-coded secret Invalid IP binding HTTP without TLS Missing default case No integrity check Weak crypto alg.	Graph-Based Static Analysis
TaintPup [10]	Puppet	Admin by default Empty password Hard-coded secret Invalid IP binding HTTP without TLS Weak crypto alg.	Syntax-Driven Taint Analysis

Saavedra et al. [8] presented GLITCH, a novel, technology-agnostic framework designed to detect security smells in IaC scripts across different technologies like Ansible, Chef, and Puppet. Unlike existing tools that focus on specific technologies, GLITCH provides a unified approach by transforming IaC scripts into an intermediate representation, enabling consistent detection of security smells. The authors conducted a large-scale empirical study using three datasets comprising 196,755 IaC scripts and 12,281,251 lines of code (LOC) to evaluate GLITCH. The tool supports the detection of nine security smells and demonstrated higher precision and recall compared to state-of-the-art tools such as SLIC and SLAC. Specifically, GLITCH achieved average precision improvements of

8-28 percentage points and recall improvements of 8-26 percentage points across different datasets.

Opdebeeck et al. [30] developed GASEL, a novel security smell detector for Ansible scripts, addressing limitations of prior tools by incorporating control-flow and data-flow analysis through Program Dependence Graphs (PDGs). GASEL, leveraging syntax awareness and inter-procedural analysis, achieved better recall and precision than SLAC and GLITCH for some smells. However, it focuses on a single IaC technology and does not implement Suspicious Comments, Hard-coded Usernames, and Missing Default in Case Statement smells detection.

Finally, Rahman et al. [10] conducted an empirical study on the propagation of security weaknesses in Puppet manifests, which are critical for managing computing infrastructure. The authors introduced TaintPup, a static analysis tool that improves the detection of security weaknesses by leveraging Puppet-specific information flow analysis. TaintPup demonstrates 2.4 times higher precision and 1.8 times higher F-measure compared to SLIC across a dataset of 17,629 Puppet manifests.

Table 2 summarizes the aforementioned studies regarding IaC technologies, security smells, and detecting approaches.

Given the landscape of research on IaC smell detection, we have chosen to focus on GLITCH. This decision is based on GLITCH's unique ability to operate across multiple IaC technologies, making it a polyglot solution. Additionally, it supports the detection of the nine security smells reported in II-B, offering a more comprehensive approach compared to other tools.

III. SecLLM

As we have seen in previous sections, security smell detection may benefit from semantic understanding to improve identification—a capability inherent to LLMs but requiring systematic orchestration to ensure reliability.

The main idea from which we conceived SecLLM is to guide LLMs through carefully engineered prompts that transform their general language understanding into specialized security analysis, while providing architectural mechanisms to manage their inherent non-determinism, as explained in the following sections.

A. KEY COMPONENTS

SecLLM is structured around four key components (see Figure 3), namely (1) *Configuration Manager*, (2) *Execution Controller*, (3) *Script Preprocessor*, and (4) *Smell Analyzer*. These components are designed to handle specific aspects of the security smell detection process and work together to provide a flexible and practical framework for analyzing IaC scripts using LLMs.

The process begins with the **Configuration Manager**, tasked with loading and managing the settings necessary for SecLLM's operation. It reads a YAML configuration file and establishes the environment parameters, including the selection of LLMs, the prompts, and the specific security

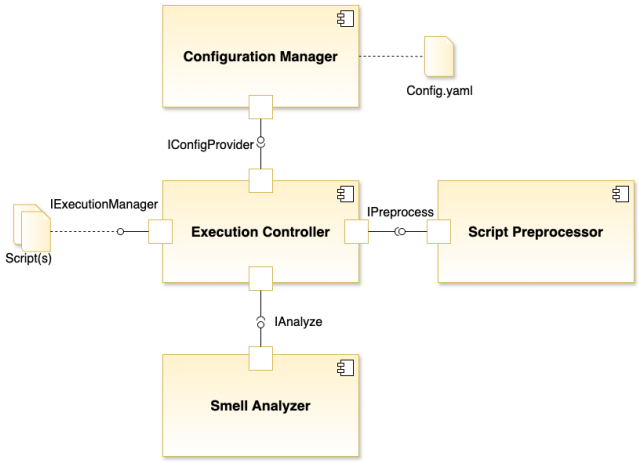

FIGURE 3. SecLLM Components. Architecture showing the four main components: Configuration Manager (setup), Execution Controller (workflow orchestration), Script Preprocessor (filtering), and Smell Analyzer (LLM-based detection with caching and ensemble support).

smells to be analyzed. This setup phase is crucial, as it ensures that the framework is initialized with the correct parameters, allowing for dynamic adjustments based on user preferences or specific project requirements.

Once the configuration is established, the **Execution Controller** takes over to orchestrate the analysis workflow. This component is responsible for managing the scheduling of tasks, ensuring that multiple security smells can be analyzed concurrently for each script. When dealing with an entire directory of scripts, the Execution Controller efficiently distributes the workload by processing multiple scripts in parallel and subsequently analyzing each script for all configured smells. This parallel execution capability significantly enhances the performance of SecLLM, enabling it to handle large datasets and complex analyses.

The **Script Preprocessor** then prepares each script for analysis. It identifies the type of IaC script—Ansible, Puppet, or Chef—and selects the appropriate prompt from the configuration. The preprocessor also optimizes the script content by filtering out non-essential elements. For instance, when identifying suspicious comments, we are interested in the commented lines, not the rest of the script; on the other hand, when looking for other smell types, comments are irrelevant and can be ignored. This preprocessing reduces the number of tokens processed by the LLMs, focusing the analysis on the most pertinent information, and improving overall accuracy and efficiency.

Finally, the **Smell Analyzer** serves as the core component for detecting security smells. It leverages the configured LLMs and uses the selected prompts and models to analyze the preprocessed script, performing a detailed analysis tailored to the specific security smell and script type. The Smell Analyzer processes the output from the LLMs to identify any detected smells, assess their severity, and pinpoint their exact location within the script. In addition

to detecting security smells, the Smell Analyzer provides developers with comprehensive feedback for each analysis.

For example, when analyzing the Puppet script in Figure 1 for hard-coded secrets, SecLLM outputs:

- Detection: *Hard-coded secret found at line 5."*
- Analysis: *Passwords 'supersecret' embedded in nested data structures ($db_configs['credentials'][1]) used at line 16 in an execution context. Replace with variable references: $db_configs['credentials'][1] = $::db_password.*

This structured output format facilitates integration with development workflows while providing actionable guidance for addressing detected issues. To enhance reliability and reproducibility, the Smell Analyzer provides three complementary mechanisms. First, it extracts not only detection results but also their confidence scores computed from the log probabilities of the LLM's output tokens. This score represents the model's certainty in its decision, enabling the framework to distinguish between high-confidence detections and uncertain cases requiring additional validation (i.e., human review). Second, the analyzer implements a caching system for LLM responses indexed by request parameters, to ensure identical results for repeated analyses of the same script-smell combinations. Third, when ensemble mode is enabled by the Execution Controller, the Smell Analyzer performs multiple independent analyses (typically five executions) of the same script-smell combination. The results are then analyzed using majority voting, where the most common detection pattern is chosen as the outcome after predictions are grouped according to detected line numbers. In case of ties, the ensemble selects the prediction with the highest average confidence score.

B. PROMPT ENGINEERING

The architectural components require prompts that guide LLMs toward accurate security smell identification and ensure structured, parseable outputs. Our approach integrates the Persona Pattern, the COSTAR framework, and Few-shot Learning, each contributing uniquely to the effectiveness of security smell detection.

The **Persona Pattern**, proposed by White et al. [32], was employed to enhance the contextual relevance of the prompts. This pattern helps the LLMs adopt a mindset conducive to identifying security smells, thereby improving the precision of the analysis.

We adopted the **COSTAR** approach to construct prompts that guide the LLMs in a structured manner [33]. COSTAR, an acronym for Context, Objective, Style, Tone, Audience, and Response, provides a comprehensive framework for prompt creation:

- Context (C): We supply the LLMs with detailed background information about the IaC environment and the specific security smells to detect, ensuring that the models understand the use case and tailor their analysis accordingly.
- Objective (O): Clear articulation of the task is provided, focusing the models on the desired outcome—accurately detecting security smells in the scripts.
- Style (S) and Tone (T): These elements are defined to ensure that the LLMs' responses are consistent with the professional and technical nature of the task, facilitating clarity and relevance in the output.
- Audience (A): The prompts assume domain expertise in cybersecurity and IaC technologies, utilizing technical terminology and concepts without requiring basic explanations, thereby enabling more precise and efficient analysis.
- Response (R): We specify the desired response format, such as text or JSON, to ensure compatibility with the analysis pipeline and facilitate seamless integration into the overall framework.

Few-shot Learning was also incorporated to enhance the model's ability to generalize from limited examples [34]. By providing a small number of illustrative instances within the prompts, we enabled the LLMs to infer patterns and apply their understanding to new, unseen scripts. This approach is particularly useful in scenarios where comprehensive datasets are unavailable, allowing the models to extrapolate from minimal input.

To ensure high-quality few-shot examples that effectively guide the LLMs without introducing bias, we implemented a systematic generation and validation process. We employed GPT-4o to generate 50 diverse IaC code snippets for each technology (Ansible, Chef, Puppet) without explicitly requesting security smells. The generation prompts requested realistic configuration scenarios covering common deployment patterns such as web server setups, database configurations, user management, and network configurations.

Two cybersecurity experts with 8+ years of experience in IaC security assessment worked independently on separate halves of the generated snippets. Each expert modified their assigned snippets to introduce specific security smells according to the GLITCH taxonomy, creating both positive examples (with smells) and negative examples (without smells). The experts then cross-validated each other's examples by independently identifying the presence and type of security smells in the examples created by their counterparts. Examples were retained only when both experts achieved perfect agreement on smell identification. For cases of disagreement, experts discussed the discrepancies and either revised the examples to achieve clarity or discarded them. We measured inter-rater agreement using Cohen's kappa, achieving $k = 0.82$ for the final retained examples, indicating strong consensus.

To further refine our selection, we eliminated redundant examples by computing cosine similarity between sentence embeddings (using OpenAI's text-embedding-3-small). Examples with similarity above 0.85 within categories were pruned, keeping only the highest-rated instance from the expert validation phase. We computed token_sort_ratio

similarity scores and semantic similarity using CodeBERT embeddings between all examples and oracle scripts to prevent overlap with oracle datasets, excluding any examples with token_sort_ratio > 0.3 or cosine similarity > 0.4.

A a result of the prompt engineering approach, we developed 25 specialized prompts, one per smell and technology, to ensure precise detection. Most smells have three prompts (one for each technology), though "Unrestricted IP Address" and "No Integrity Check" share prompts between Chef and Puppet due to their similar patterns. The complete set of prompts can be accessed in our replication package [35] (see `config.yaml`). All prompts have the same COSTAR sections, but comprise the instructions and examples specific to the smells and IaC technologies to analyze. Figure 4 provides an example of the prompt template used for detecting the "Suspicious Comment" smell in Ansible scripts, illustrating how the described techniques are applied:

- **CONTEXT**: This section establishes the LLM's role as an "IaC Security Expert" using the Persona Pattern, creating appropriate analytical framing before presenting the target script.
- **OBJECTIVE**: It provides explicit detection criteria through the COSTAR framework: (a) searching for predefined keywords indicating defects or future tasks, (b) excluding URLs containing the keywords and documented workarounds, and (c) identifying issues yet to be resolved, without temporary workarounds.
- **RESPONSE FORMAT**: It defines structured output with `<comment_analysis>` tags containing line numbers, identified keywords, reasoning chains, and conclusions—enabling reliable parsing and traceability of detection decisions.
- **FEW-SHOT EXAMPLES**: This section includes three curated examples, two demonstrating true positive identification with explicit reasoning (e.g., "TODO: Fix…" indicating future task), and one providing a negative case handling (advisory comments without actionable defects). Each example shows the complete input-analysis-output cycle, teaching the LLM to distinguish between problematic patterns and legitimate documentation.

It is worth noting that the response format also contains instructions for reporting a detailed analysis of the given script.

C. FLEXIBILITY AND EXTENSIBILITY

SecLLM's design enables adaptation to evolving security threats, emerging IaC technologies, and varying organizational requirements through several extensibility mechanisms.

First of all, the framework facilitates expansion to new IaC technologies through technology-specific prompt templates and configuration entries alone, without modifying the core pipeline. New security smell patterns can be integrated by defining detection prompts and updating the YAML configuration file.

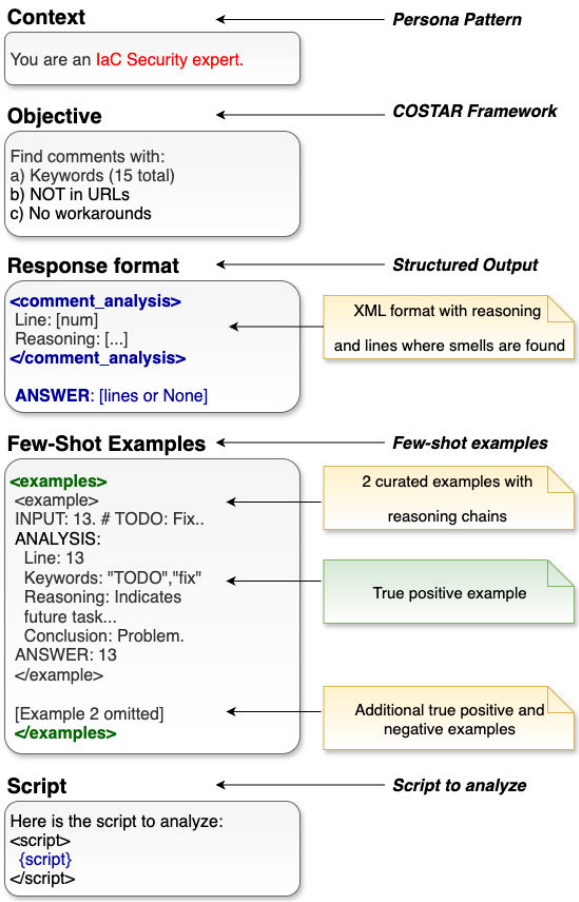

FIGURE 4. Suspicious comment detection prompt template (abbreviated).

Second, SecLLM can be extended to support unseen IaC technologies by leveraging LLMs' ability to generalize beyond training data. According to recent research, LLMs can handle novel syntactic structures by transferring semantic understanding across languages through abstract pattern recognition [36], [37] and computational similarities [38]. This feature works particularly well for IaC technologies that share declarative abstractions (conditionals, dependencies, and resource definitions) across syntaxes. SecLLM exploits this capability through few-shot prompting (Section III-A), where minimal examples provide syntactic grounding while LLMs apply broader security reasoning. When generalization is insufficient, domain-fine-tuned models, retrieval-augmented generation (e.g., implementing new filters in the Script Preprocessor), and ensemble voting (Section III-A) can be employed to combine models with complementary training exposures or enrich few-shot examples with technology-specific patterns.

Third, the framework's model-agnostic design allows organizations to evaluate emerging LLMs and leverage improvements in language understanding or cost efficiency by simply updating configuration parameters.

Finally, another important aspect is related to the analysis customization through modular prompt engineering. Indeed, teams can adjust detection sensitivity, integrate organization-specific coding standards, or modify analysis verbosity to meet their operational requirements. Additionally, SecLLM facilitates hybrid deployment strategies by assigning distinct LLMs to particular security smells according to their computational profiles or detection strengths. This granular control optimizes both detection accuracy and operational expenses across infrastructure analysis workflows.

D. SecLLM IMPLEMENTATION

SecLLM is implemented in Python with a YAML-driven configuration system.

The configuration file stores LLM endpoint configurations, prompt templates, smell-specific detection parameters, and execution settings (e.g., caching, retry, parallelism).

The framework supports several LLM providers (OpenAI, Anthropic, and open-weights alternatives) by standardizing request/response handling and credential management through a single interface. Configurable exponential backoff retry policies allow handling transient failures, network interruptions, and rate limiting of LLM calls. Failed analyses are logged without blocking subsequent processing, ensuring that partial results are always preserved.

To reduce analysis costs and latency, SecLLM implements a persistent disk-based cache that indexes LLM responses using SHA-256 hashes of request parameters. Cache entries are serialized in pickle format with configurable Time-To-Live and automatic expiration handling.

In terms of processing, the framework uses two-level parallelization through `ThreadPoolExecutor`: (i) processing multiple scripts at once when examining directories, and (ii) detecting multiple smells within individual scripts at once.

Lastly, the analysis results can be exported to JSON and include the execution time, token consumption, and detected smells (along with a description, affected lines, justification, and confidence score) for each analyzed file. This feature makes it easy to integrate SecLLM with CI/CD pipelines and security dashboards. The Python sources are publicly available in the SecLLM online repository [35].

IV. EXPERIMENTAL EVALUATION

This section presents the empirical evaluation of SecLLM across three dimensions: detection accuracy, computational feasibility, and deterministic behavior. The evaluation assesses whether the framework's prompt engineering, architectural mechanisms, and implementation choices meet the accuracy, cost-effectiveness, and reliability requirements for practical deployment.

A. RESEARCH GOALS AND QUESTIONS

The empirical assessment addresses three research questions:

RQ1 [**Accuracy and Performance**] How does SecLLM compare to existing state-of-the-art tools for detecting security smells in terms of accuracy and performance?

RQ2 [**Computational Feasibility**] What are the token consumption patterns and associated costs of SecLLM, and is LLM-based security smell detection economically viable for practical deployment?

RQ3 [**Determinism and Reliability Protocol**] To what extent does SecLLM exhibit deterministic behavior, and how do different mitigation strategies affect consistency and detection accuracy?

TABLE 3. Composition of oracle datasets.

IaC Technology	Total Scripts	Identified Smells	Scripts with No Smells
Ansible	81	44	69
Chef	80	105	43
Puppet	80	65	52

B. DATASETS

For the evaluation of SecLLM, we employed the same three oracle datasets utilized by GLITCH [8], each corresponding to a different IaC technology: Ansible, Chef, and Puppet. Saavedra et al. created the ground truth for these datasets involving a meticulous selection and annotation methodology [8]. They annotated these datasets using a closed coding process, in which multiple raters with expertise in IaC and cybersecurity independently identified security smells. To ensure reliability, only classifications where at least two raters agreed were retained. This rigorous process resulted in high agreement rates: 99.1% for Ansible, 93.9% for Chef, and 95.1% for Puppet.

Regarding specific numbers, the Ansible dataset (81 scripts) features 44 identified security smells across various categories, with 69 scripts containing no smells. The Chef dataset (80 scripts) includes 105 identified security smells, with 43 scripts free of smells. Similarly, the Puppet dataset (80 scripts) comprises 65 identified security smells and 52 scripts without any smells.

Table 3 reports the oracle datasets composition.

C. LLMs SELECTION

We employed three different LLMs to evaluate the proposed framework: GPT-4o-mini, Qwen-2.5 (14B parameters), and Qwen-2.5 (32B parameters) [39]. Our selection strategy was designed to assess method performance across different model architectures, capabilities, and deployment scenarios for privacy concerns.

GPT-4o-mini was accessed through OpenAI's API. We opted to use GPT-4o-mini because it represents a cost-effective cloud solution that maintains strong performance capabilities while offering significantly reduced computational costs compared to larger models.

The inclusion of both Qwen-2.5 variants (14B and 32B parameters) represents a strategic decision to evaluate our methods with open-source alternatives that offer different architectural characteristics and computational requirements.

Both models are decoder-only transformers that provide substantial capacity for complex reasoning tasks while remaining deployable in local environments to preserve privacy. The 14B parameter model offers a more lightweight option suitable for resource-constrained environments, while the 32B parameter model provides enhanced capabilities for more complex analysis tasks. Both Qwen-2.5 models were deployed locally using LM Studio, a comprehensive platform for LLM experimentation that provides a unified OpenAI-compatible REST API for model deployment and inference across various architectures.

D. PRELIMINARY ANALYSIS: ADDRESSING POTENTIAL DATA LEAKAGE

Before conducting the main empirical evaluation, we carried out a preliminary analysis to investigate potential data leakage concerns, given that LLMs might have encountered IaC scripts during their training phase.

Recent research on LLM training dynamics has established that while these models may memorize portions of their training data, their ability to perform specialized tasks is far more nuanced. Studies by Chang et al. [40] and Inaba et al. [41] demonstrate that LLMs acquire knowledge gradually through their architecture and training dynamics, developing general capabilities rather than specialized task performance. Critically, Al-Kaswan et al. [42] found that even when LLMs exhibit traces of memorization in code, such memorization does not necessarily translate to high performance on specific code analysis tasks. Leybzon et al. [43] further demonstrated that without targeted training on specific tasks, LLMs often fail to apply memorized information to specialized applications effectively. Based on these theoretical foundations, even if LLMs encountered IaC scripts in their training data, they would likely lack the specialized capability to identify security smells without proper guidance accurately. SecLLM provides such guidance through carefully engineered prompts that direct the model to apply its general knowledge to the specific task of security smell detection.

To empirically address the data leakage concern, we conducted four experiments, using the LLMs reported in Section IV-C, that provide evidence that our results are not influenced by memorization of the GLITCH dataset:

- **Experiment 1: GLITCH Knowledge Assessment**
  We evaluated LLMs' knowledge of the GLITCH tool itself by prompting:
  "What is GLITCH? When has GLITCH been published?"
  The model demonstrated no knowledge of the GLITCH smell detection tool or research paper, instead describing completely unrelated concepts:
  "GLITCH is an online storytelling platform that combines interactive narratives and gaming elements…"
- **Experiment 2: Script Content Memorization Test**
  For all scripts in our evaluation dataset, we prompted:
  "Report the content of the following script name. Script name: {script}"
  The model consistently responded:
  "I'm unable to access specific script contents directly…"
  Results confirm the model cannot recall script contents by name alone.
- **Experiment 3: Oracle Results Memorization Test**
  We tested for memorization of the GLITCH oracle results by prompting:
  "The tool GLITCH can identify the following smells: Admin by default, Empty password, Hard-coded secret, Unrestricted IP Address, Suspicious comment, Use of HTTP without SSL/TLS, Use of weak cryptography algorithms, No integrity check, Missing Default in Case Statement. I'll provide the script name and you will report for each smell the comma-separated list of lines where it occurs or none if it doesn't occur. Script name: {script}"
  The LLMs consistently responded: *"Sure! Please provide the script content…"* indicating no memorization of the oracle results.
- **Experiment 4: Direct Smell Detection Performance Test**
  As a definitive test, we evaluated whether LLMs could accurately identify security smells when provided with the actual script content without our specialized prompting technique. We calculated standard performance metrics for each smell type across all three IaC languages, with results showing extremely poor performance:
  - Most smells showed precision, recall, and F1-scores of 0 or near 0
  - The highest F1-score across all languages was only 0.19
  - For Chef scripts, the best performance was an F1-score of 0.10
  - For Puppet scripts, the best was an F1-score of 0.15 for "Suspicious comment"

These results align with the theoretical understanding from recent literature. The stark performance gap between direct smell detection (very poor) and the SecLLM approach (significantly better) provides evidence that our results stem from effective prompt engineering rather than from data leakage.

E. SETUP AND TESTING PROTOCOL

To ensure reproducibility and reliability of the evaluation, we proceeded as follows. All experiments were conducted using a MacBook M3 Max with 14 cores and 96 GB RAM. To optimize deterministic behavior within the limitations of API-based inference, each LLM was configured with a temperature parameter set to 0 and top_p set to 1.0.

The experimental workflow was orchestrated through a shell script (`analysis.sh`) that processed each oracle dataset provided by GLITCH (see Section IV-B). We passed the oracle-dataset path, the JSON output file for storing results, and the configuration file to use (i.e., we provided one YAML file for each LLM) to the shell script. For each IaC script in the dataset, the shell script invoked SecLLM with parallel smell detection enabled, analyzing all nine security smells concurrently. Only for Qwen-2.5 models, considering their substantial computational resource requirements during local inference, we reduced parallelism to 3 concurrent smell analyses to maintain system stability and prevent memory saturation.

As described in Section III-D, SecLLM automatically saved analysis results in JSON format with append mode enabled, in order to accumulate in the same file the smell analyses for each IaC technology. This design choice facilitated the collection of data across independent runs without manual intervention or risk of data loss between executions.

A hierarchical aggregation approach was used in the analysis process. Metrics were computed at the individual smell level for each script using custom Python scripts that processed the JSON files described above. These metrics were then aggregated for all scripts in each dataset.

F. RESEARCH METHOD FOR RQ_1

To address RQ_1, we ran five times SecLLM for the oracle datasets described in section IV-B. We decided to perform five executions due to the non-determnistic nature of LLMs, that we specifically assessed in RQ_3 (see Section IV-H).

We then measured the accuracy of SecLLM by comparing its outputs against the ground truth established in the oracle datasets, calculating precision, which is the proportion of true positive detections out of all positive detections made by SecLLM, and recall, which measures the proportion of true positive detections out of all actual smells present in the datasets. We calculated also F1-score, which is the harmonic mean of precision and recall, providing a single metric that balances both false positives and false negatives. The F1-score is particularly useful in scenarios with uneven class distribution, where merely focusing on accuracy could be misleading.

To ensure statistical significance and account for variability across different runs, we averaged the above-mentioned metrics from the five executions and computed standard deviations for each of them.

G. RESEARCH METHOD FOR RQ_2

The RQ_2 goal is to addresses economic viability of the proposed framework. To this aim, during the execution of the analyses reported in RQ_1 (see Section IV-F), we calculated both token consumption patterns and associated classification costs. For token usage analysis, we tracked the total number of input and output tokens required by each LLM to process every script in our test sets.

To translate token usage into practical cost considerations, we adopted a token-based accounting approach grounded in the pricing models of commercial LLM-as-a-service offerings. Token counts were multiplied by reference per-token costs to derive cost estimates that reflect real-world deployment scenarios. The reference costs for GPT-4o-mini were obtained from OpenAI's publicly available pricing for their language model service offerings.[4] For Qwen-2.5, which was deployed locally during our experiments, we simulated a realistic cloud-based deployment scenario by applying the pricing model offered by Cloudflare Workers AI.[5] This approach ensures fair and consistent cost comparisons across all models.

The reference rates (in USD) we considered are:

- GPT-4o-mini: $ 0.15 per 1M input tokens, $ 0.60 per 1M output tokens;
- Qwen-2.5 14B: $ 0.35 per 1M input tokens, $ 1.40 per 1M output tokens;
- Qwen-2.5 32B: $ 0.70 per 1M input tokens, $ 2.80 per 1M output tokens.

H. RESEARCH METHOD FOR RQ_3

RQ_3 aims at investigating strategies to quantify, mitigate, and manage the inherent non-determinism of LLMs and establish practical guidelines for practitioners. We structured the investigation into three phases.

1) BASELINE NON-DETERMINISM CHARACTERIZATION

We began by quantifying the extent and patterns of non-deterministic behavior analyzing the results of the five runs performed to answer RQ_1. In our study, we assessed the reliability of smell detection outcomes by considering each experimental repetition as a separate evaluator and determining the level of inter-rater agreement among these different detections. To achieve this, we employed two metrics widely recognized in Software Engineering research, such as those discussed in [44]: the raw percentage of agreement and the Fleiss' Kappa statistic. The raw percentage of agreement is calculated as the proportion of smell detections where all experimental iterations assigned the same label, as referenced in [44]. This metric offers a clear summary of smell detection consistency across various executions of SecLLM, yet it does not account for the likelihood of chance agreements. Therefore, we also calculate the Fleiss' Kappa agreement index, as outlined in [45]. Fleiss' Kappa provides a statistical evaluation of agreement among multiple raters and considers the likelihood of random agreements. The score ranges from 0 to 1, where 1 denotes perfect agreement and 0 indicates that any agreement is purely due to chance, consistent with the definitions provided in [46].

[4] OpenAI Pricing. https://platform.openai.com/docs/pricing

[5] Alibaba Cloud Models and Pricing. https://www.alibabacloud.com/help/en/model-studio/models

2) MITIGATION STRATEGY EVALUATION

In the second phase, we evaluated three complementary strategies designed to address non-determinism, namely confidence-based filtering, ensemble-based consensus, and response caching.

Confidence-Based Filtering. For each prediction, we obtained by LLMs the confidence score via log probabilities as

$$confidence = \exp\left(\frac{1}{n}\sum_{i=1}^{n}\log p_i\right) \tag{1}$$

where $\log p_i$ represents the log probabilities of output tokens provided by the models. We stratified results into correct and incorrect predictions, then applied Wilcoxon Rank-Sum Tests to evaluate whether confidence values significantly differ between these groups. We employed Holm-Bonferroni correction with $\alpha = 0.05$ to account for multiple comparisons across LLM-technology combinations.

We assessed precision and recall at confidence cutoff points ranging from 0.5 to 1.0 in 0.05 increments in order to determine useful intervention thresholds. For each threshold, we calculated precision and recall to identify the optimal cutoff point that maximizes F1-score, which strikes the best balance between the requirement for human oversight and the accuracy of automated analysis. This cutoff point shows the degree of confidence at which forecasts ought to be marked for manual examination.

Ensemble-Based Consensus. We employed the ensemble mechanism of the Execution Controller (see Section III-A), using five Smell Analyzer instances, and running an independent execution for each IaC oracle dataset. We choose to have five instances in the ensemble because, despite possible outliers or residual non-determinism in the smell identification process, this configuration reduces the possibility of ties by guaranteeing a clear majority vote (at least three out of five) in the majority of cases. We compared ensemble predictions against the results obtained in RQ1 by measuring: (i) detection accuracy improvements through precision, recall, and F1-score differences; (ii) determinism gains through Fleiss' Kappa and raw percentage agreement compared to five independent single runs; and (iii) computational cost as the 5x multiplicative factor in API calls and inference time.

Response Caching. We validated the Smell Analyzer caching mechanism that stores LLM responses keyed by request parameters. We executed five runs with caching enabled and measured Fleiss' Kappa and raw percentage agreement to verify perfect reproducibility. To verify that caching preserved detection accuracy while removing variability across executions, we also calculated and compared precision, recall, and F1-scores with the baseline results from RQ1.

3) COMBINED STRATEGY ANALYSIS

In the last phase, we evaluated four configurations to determine optimal deployment strategies, namely baseline single-run execution (i.e., the best run among the five ones performed in RQ1), confidence filtering only, ensemble voting only, and caching enabled only. We calculated precision, recall, and F1-score for performances, Fleiss' Kappa for determinism, and total token consumption for each configuration. In order to determine which combination offers the best trade-off, we then compared these metrics across configurations. The decision criteria were developed through multi-dimensional trade-off analysis, evaluating each configuration across four dimensions: (i) detection accuracy (precision, recall, F1-score), (ii) determinism (Fleiss' Kappa and raw agreement), (iii) computational cost (token consumption multiplier), and (iv) deployment constraints (privacy requirements). Configurations were considered effective if they improved determinism (Δ Kappa $\geq$ 0.05) while maintaining accuracy within 2% of baseline performance ($|\Delta F1| \leq 0.02$) or provided other strategic benefits (e.g., cost reduction, privacy compliance).

V. RESULTS, DISCUSSION, AND IMPLICATIONS

We present and discuss the results achieved for each research question, and elaborate on their implications and the framework's limitations.

A. RESULTS AND DISCUSSION

In the following, we report the achieved results for RQ_1, RQ_2, and RQ_3.

1) RQ_1-ACCURACY AND PERFORMANCE

Tables 4, 5, and 6 present a comparison between SecLLM (using three different models: GPT-4o-mini, Qwen-2.5 14B, and Qwen-2.5 32B) and GLITCH across Ansible, Chef, and Puppet datasets.

The Ansible dataset (Table 4) demonstrates SecLLM's superior performance across all model variants. Qwen-2.5 32B achieved the best overall results with perfect precision (1.00, SD=0.00) and near-perfect recall (0.99, SD=0.00), outperforming GLITCH's average precision of 0.76 and recall of 0.86. A particular improvement is observed in "Hard-coded secret" detection, where Qwen-2.5 32B achieved perfect precision (1.00) compared to GLITCH's poor precision of 0.42, though GLITCH maintained perfect recall. The "No integrity check" pattern represents a critical advantage for SecLLM, as all variants achieved perfect scores while GLITCH completely failed to detect any instances. Additionally, all SecLLM models maintained high precision (0.94-0.99) for "No smell" classification, demonstrating their ability to minimize false positives.

The Chef dataset (Table 5) presents a more challenging scenario with a broader range of security smells across 148 total instances. Qwen-2.5 32B again leads with 0.92 precision (SD=0.00) and 0.93 recall (SD=0.00), compared to GLITCH's 0.77 precision and 0.86 recall. The "Admin by default" pattern shows interesting performance trade-offs, where GLITCH achieved higher precision (0.94) but significantly lower recall (0.41). At the same time, SecLLM

TABLE 4. SecLLM vs GLITCH: Accuracy for the ansible datasets (N/A - Not applicable, N/D - No data).

		SecLLM Baseline For Ansible							
		GPT-4o-mini		Qwen-2.5 14B		Qwen-2.5 32B		GLITCH	
Smell	Occurr.	Prec. (SD)	Recall (SD)	Prec. (SD)	Recall (SD)	Prec. (SD)	Recall (SD)	Prec.	Recall
Admin by default	7	**1.00** (0.00)	**1.00** (0.00)	**1.00** (0.00)	0.86 (0.00)	**1.00** (0.00)	**1.00** (0.00)	**1.00**	**1.00**
Empty password	1	**1.00** (0.00)	**1.00** (0.00)	**1.00** (0.00)	**1.00** (0.00)	**1.00** (0.00)	**1.00** (0.00)	**1.00**	**1.00**
Hard-coded secret	8	0.95 (0.06)	0.80 (0.06)	0.92 (0.07)	0.78 (0.05)	**1.00** (0.00)	0.88 (0.00)	0.42	**1.00**
Invalid IP address binding	1	**1.00** (0.00)	**1.00** (0.00)	**1.00** (0.00)	**1.00** (0.00)	**1.00** (0.00)	**1.00** (0.00)	**1.00**	**1.00**
Suspicious comment	6	**1.00** (0.00)	**1.00** (0.00)	0.77 (0.04)	**1.00** (0.00)	**1.00** (0.00)	**1.00** (0.00)	0.75	**1.00**
Use of HTTP without TLS	20	**1.00** (0.00)	**1.00** (0.00)	**1.00** (0.00)	0.95 (0.00)	**1.00** (0.00)	**1.00** (0.00)	0.95	**1.00**
No integrity check	1	0.90 (0.20)	**1.00** (0.00)	**1.00** (0.00)	**1.00** (0.00)	**1.00** (0.00)	**1.00** (0.00)	0.00	0.00
Use of Weak crypto alg.	0	N/D	N/D	N/D	N/D	N/D	N/D	N/D	N/D
Missing default case statement	0	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
No smell	69	0.98 (0.10)	0.99 (0.00)	0.95 (0.01)	0.95 (0.01)	**0.99** (0.00)	**1.00** (0.00)	0.98	0.94
Average		0.98 (0.03)	0.97 (0.01)	0.95 (0.02)	0.94 (0.01)	**1.00** (0.00)	**0.99** (0.00)	0.76	0.86

TABLE 5. SecLLM vs GLITCH: Accuracy for the chef datasets.

		SecLLM Baseline For Chef							
		GPT-4o-mini		Qwen-2.5 14B		Qwen-2.5 32B		GLITCH	
Smell	Occurr.	Prec. (SD)	Recall (SD)	Prec. (SD)	Recall (SD)	Prec. (SD)	Recall (SD)	Prec.	Recall
Admin by default	37	0.85 (0.03)	0.91 (0.02)	0.83 (0.00)	0.81 (0.00)	0.85 (0.00)	**0.92** (0.00)	**0.94**	0.41
Empty password	4	0.84 (0.08)	**1.00** (0.00)	**1.00** (0.00)	**1.00** (0.00)	**1.00** (0.00)	**1.00** (0.00)	**1.00**	0.75
Hard-coded secret	13	0.76 (0.06)	**0.77** (0.00)	0.79 (0.03)	**0.77** (0.00)	**0.81** (0.09)	**0.77** (0.00)	0.20	0.69
Invalid IP address binding	7	**1.00** (0.00)	**1.00** (0.00)	**1.00** (0.00)	**1.00** (0.00)	**1.00** (0.00)	**1.00** (0.00)	**1.00**	**1.00**
Suspicious comment	4	0.96 (0.08)	**1.00** (0.00)	0.84 (0.08)	**1.00** (0.00)	**1.00** (0.00)	**1.00** (0.00)	0.40	**1.00**
Use of HTTP without TLS	13	**0.86** (0.00)	**0.92** (0.00)	**0.86** (0.00)	**0.92** (0.00)	**0.86** (0.00)	**0.92** (0.00)	0.71	**0.92**
No integrity check	6	**1.00** (0.00)	**1.00** (0.00)	**1.00** (0.00)	**1.00** (0.00)	0.86 (0.00)	**1.00** (0.00)	**1.00**	**1.00**
Use of Weak crypto alg.	1	**1.00** (0.00)	**1.00** (0.00)	**1.00** (0.00)	**1.00** (0.00)	**1.00** (0.00)	**1.00** (0.00)	0.50	**1.00**
Missing default case statement	20	0.89 (0.02)	**0.96** (0.02)	**1.00** (0.00)	0.85 (0.00)	0.90 (0.00)	0.90 (0.00)	**1.00**	0.95
No smell	43	0.84 (0.01)	0.74 (0.03)	0.84 (0.00)	0.81 (0.00)	0.87 (0.00)	0.78 (0.00)	**0.95**	0.88
Average		0.90 (0.03)	**0.93** (0.01)	**0.92** (0.01)	0.92 (0.00)	**0.92** (0.00)	**0.93** (0.00)	0.77	0.86

balanced both metrics more effectively, with precision ranging from 0.83-0.85 and recall from 0.81-0.92 across all LLMs. Substantial improvements in precision-critical scenarios are evident, particularly for "Hard-coded secret" detection, where SecLLM achieved precision values between 0.71 and 0.77 compared to GLITCH's 0.20. Similarly, "Suspicious comment" detection showed SecLLM's superiority with precision values ranging from 0.80 to 1.00 compared to GLITCH's 0.40. The "Missing default case statement" pattern demonstrated competitive performance across all approaches, with both SecLLM and GLITCH achieving high precision (0.89-1.00), indicating that this particular smell's detection complexity is manageable across different methodologies.

Regarding the Puppet dataset (Table 6), SecLLM achieved superior average performance with Qwen-2.5 32B, leading at 0.89 precision and 0.90 recall (SD=0.00), compared to GLITCH's 0.68 precision and 0.85 recall. The "Hard-coded secret" pattern remains challenging across all approaches, with all methods showing lower precision, but SecLLM variants (0.57-0.75) substantially outperformed GLITCH (0.14). Perfect detection capabilities were demonstrated by SecLLM for "Invalid IP address binding" and "No integrity check," with GLITCH showing no available data for the latter.

a: CROSS-MODEL PERFORMANCE ANALYSIS

Concerning the cross-model analysis, Qwen-2.5 32B emerged as the top performer across all datasets, achieving the highest average precision and recall scores. The comparison between Qwen-2.5 14B and 32B variants shows that the larger model provides more consistent performance across different security smell types, particularly excelling in complex patterns like "Hard-coded secret" detection. Notably, GPT-4o-mini demonstrated competitive performance despite different architectural foundations, often matching or closely approaching the Qwen variants' results.

This outcome suggests that the LLM-based approach itself is robust across different model architectures, rather than being dependent on a specific implementation. All SecLLM LLMs (F1-score: 0.87-0.94) substantially outperformed GLITCH (F1-score: 0.63-0.77) across all datasets,

TABLE 6. SecLLM vs GLITCH: Accuracy for the puppet datasets (N/D - No data).

		SecLLM Baseline For Puppet							
		GPT-4o-mini		Qwen-2.5 14B		Qwen-2.5 32B		GLITCH	
Smell	Occurr.	Prec. (SD)	Recall (SD)	Prec. (SD)	Recall (SD)	Prec. (SD)	Recall (SD)	Prec.	Recall
Admin by default	14	0.84 (0.04)	**1.00** (0.00)	0.82 (0.00)	**1.00** (0.00)	**1.00** (0.00)	0.71 (0.00)	0.81	0.93
Empty password	5	0.87 (0.11)	**1.00** (0.00)	0.83 (0.00)	**1.00** (0.00)	**1.00** (0.00)	**1.00** (0.00)	**1.00**	**1.00**
Hard-coded secret	11	0.57 (0.03)	0.89 (0.04)	0.59 (0.03)	0.82 (0.00)	0.75 (0.00)	0.82 (0.00)	0.14	0.82
Invalid IP address binding	6	**1.00** (0.00)	**1.00** (0.00)	**1.00** (0.00)	**1.00** (0.00)	**1.00** (0.00)	0.83 (0.00)	**1.00**	**1.00**
Suspicious comment	9	0.87 (0.09)	0.93 (0.05)	**1.00** (0.00)	0.89 (0.00)	0.90 (0.00)	**1.00** (0.00)	0.39	**1.00**
Use of HTTP without TLS	5	**0.80** (0.00)	0.80 (0.00)	**0.80** (0.00)	0.80 (0.00)	**0.80** (0.00)	0.80 (0.00)	0.45	**1.00**
No integrity check	1	**1.00** (0.00)	**1.00** (0.00)	**1.00** (0.00)	**1.00** (0.00)	**1.00** (0.00)	**1.00** (0.00)	N/D	0
Use of Weak crypto alg.	4	0.70 (0.09)	**1.00** (0.00)	**1.00** (0.00)	**1.00** (0.00)	0.80 (0.00)	**1.00** (0.00)	0.57	**1.00**
Missing default case statement	10	0.78 (0.07)	0.79 (0.02)	0.95 (0.10)	0.68 (0.10)	0.75 (0.00)	0.90 (0.00)	**0.83**	**1.00**
No smell	52	0.90 (0.02)	0.76 (0.08)	0.90 (0.01)	0.84 (0.02)	0.88 (0.00)	**0.89** (0.00)	**0.97**	0.71
Average		0.83 (0.05)	**0.92** (0.02)	0.86 (0.01)	0.89 (0.01)	**0.89** (0.00)	0.90 (0.00)	0.68	0.85

with performance improvements being most pronounced in precision-critical scenarios such as "Hard-coded secret" and "Suspicious comment" detection, where traditional rule-based approaches appear to struggle with contextual understanding.

b: ERROR ANALYSIS

To assess SecLLM's limits, we conducted an error analysis of cases where GLITCH showed better performance than SecLLM and false positives to identify the underlying causes.

Detection Limitations: GLITCH Advantage Cases. The error analysis identified 25 specific under-performance cases, categorized into 11 precision and 14 recall disadvantages, which represent approximately 2.2% of total detections (378 code smells x 3 LLMs = 1,134 total detection instances). The identified patterns display three distinct categories of detection challenges.

First, SecLLM may struggle to distinguish between legitimate code configurations and actual security vulnerabilities when they appear similar. This challenge manifests primarily in the three recall cases of the "Hard-coded secret" detection, where SecLLM implements a sophisticated disambiguation strategy that examines the administrative context of username-password pairs to prevent overlap with admin-by-default classifications. Similar disambiguation challenges occur in "Admin by default" detection (3 precision cases, 2 recall cases), where SecLLM's contextual analysis occasionally differs from GLITCH's pattern-based classifications. Specifically, SecLLM identifies usernames as hard-coded secrets only when explicitly coupled with hard-coded passwords for administrative accounts, or when occurring independently for non-administrative contexts. In contrast, GLITCH applies pattern matching indiscriminately, creating substantial overlap between admin-by-default and hard-coded secret categories, as evidenced by GLITCH's 151 false positives compared to SecLLM's 30 across all models.

Second, GLITCH shows some benefits when it is necessary to check every part of the code. This is evident in "Missing default case statement" detection (4 precision cases, 4 recall cases), where GLITCH's rule-based approach examines all switch statements to identify those missing default cases. Similar coverage advantages appear in "Use of HTTP without SSL/TLS" (4 recall cases) and "Suspicious comment" (1 recall case) detection. In contrast, SecLLM's contextual analysis focuses on semantically prominent patterns, resulting in disadvantages across these pattern types.

Third, rule-based detection maintains some advantages in specific pattern-matching scenarios. In the three precision cases of 'Empty password' detection where differences emerged, GLITCH's exact string matching approach avoids the occasional false positives that SecLLM generates. A similar precision advantage occurs in "No integrity check" detection (1 case).

The cross-model analysis reveals that these limited disadvantages occur consistently across all models. "Missing Default in Case Statement" appears most frequently across both precision (4 cases) and recall (4 cases) disadvantages. In comparison "Hard-coded secret" detection shows the most significant individual recall losses (3 cases). These patterns confirm that rule-based detection maintains narrow advantages primarily in exhaustive enumeration scenarios and exact pattern matching. On the other hand, SecLLM's contextual approach delivers superior overall performance across the vast majority of detection cases.

False Positives Analysis. Regarding false positives, we focused on those produced by the 'Hard-coded secret' detection, which has the lowest precision across Chef and Puppet technologies. We found 18 instances that can be categorized into three categories.

About 50% stem from context-dependent string literals, namely values that look like secrets but are not. A typical example is the assignment `machine_options['ssh_username'] = 'ubuntu'`, where the username "ubuntu" is simply a default in many cloud environments,

TABLE 7. Confidence scores for false positive Hard-coded secret detections.

Model	Technology	Mean Confidence (Range)
GPT-4o-mini	Chef	0.939 (0.922–0.966)
	Puppet	0.926 (0.853–0.964)
Qwen-2.5 14B	Chef	0.946 (0.925–0.957)
	Puppet	0.952 (0.916–0.977)
Qwen-2.5 32B	Chef	0.893 (0.865–0.948)
	Puppet	0.937 (0.917–0.977)
Correct predictions		*>0.973*

despite initially appearing sensitive. An additional illustration concerns test scripts that use placeholder values, such as `-os-password a_big_secret`, which, although harmless, cause the detector to signal them.

Next, variable indirection was responsible for about 30% of the errors. Instructions like `node['zabbix'] ['database'] ['dbpassword']` are flagged as a security smell, even though the actual password is stored securely elsewhere and not hard-coded.

Finally, about 20% of the false positives were due to classification ambiguity, which is essentially caused by overlapping definitions of smells. For example, lines such as `default[:mongodb][:username] = ""` (an empty username) are flagged as secrets simply because they fit the surface pattern.

The confidence scores tell a fairly consistent story (see Table 7). For false positives, confidence ranges from 0.853 to 0.977, while for correct predictions, it is higher, with a mean confidence above 0.973.

This separation suggests that confidence scores can be a reliable means for identifying dubious detections. By establishing a confidence threshold of 0.96, we can automatically discard about 83% of the false positives (15 out of 18) without losing any real detections. Doing so would raise Chef's precision from roughly 0.71–0.81 to about 0.91–0.95, and from 0.57-0.75 to 0.78-0.92 for Puppet, without requiring manual cleanup. The few remaining false positives above that cutoff appear to be genuine edge cases that may only require a few more specific examples (few-shot examples) to resolve.

These patterns point to the real difficulty of semantic analysis in configuration code. The problem is not that LLMs cannot tell secrets from non-secrets; it is that context matters, and the context is often buried elsewhere in the system. Similarly, the model must track references across files or scripts due to variable indirection, which is still challenging. So, rather than seeing these as fundamental flaws of the LLM approach, it is probably more accurate to view them as areas where targeted fine-tuning could make a big difference, especially by adding a few more instructive examples and leveraging confidence-based filtering. We explore these ideas further in Section IV-H.

Given the results, we can answer RQ_1 as follows:

Answer to RQ_1

SecLLM outperforms GLITCH across all datasets and model architectures. Qwen-2.5 32B delivers the highest accuracy, while all SecLLM variants demonstrate robust detection capabilities across diverse IaC technologies and security smell types.

TABLE 8. SecLLM token usage and cost analysis per script.

Model	Avg Input Tokens	Input Cost ($)	Avg Output Tokens	Output Cost ($)	Total Cost ($)
GPT-4o-mini	17,140	0.0026	596	0.0004	**0.0030**
Qwen-2.5 14B	17,834	0.006	741	0.001	0.007
Qwen-2.5 32B	18,505	0.013	860	0.002	0.015

2) RQ_2-COMPUTATIONAL FEASIBILITY

Regarding the computational costs and token consumption across all three models, Table 8 presents the average token usage and associated costs per script analysis.

GPT-4o-mini demonstrated the most cost-effective performance with an average of 17,140 input tokens and 596 output tokens per script, resulting in total costs of $0.0030 per script analysis. Qwen-2.5 14B required 17,834 input tokens and 741 output tokens at $0.007 per script, while Qwen-2.5 32B consumed 18,505 input tokens and 860 output tokens at $0.015 per script. These results indicate that despite Qwen-2.5 32B's superior accuracy performance, GPT-4o-mini offers the most economical solution for large-scale deployments, with costs approximately 2.3x and 5x lower than Qwen-2.5 14B and 32B respectively. The relatively consistent input token consumption across models (17,140-18,505 tokens) suggests similar context requirements, while the variation in output tokens (596-860) reflects different model verbosities in security analysis reporting.

Given the results, we can answer RQ_2 as follows:

Answer to RQ_2

SecLLM demonstrates economically viable token consumption patterns across all evaluated models, with per-script costs ranging from $0.003 to $0.015. The framework enables organizations to balance accuracy and cost based on their specific requirements, making LLM-based security smell detection feasible for both small-scale and enterprise-level deployments. GPT-4o-mini provides the most cost-effective solution, while Qwen-2.5 32B offers superior accuracy at moderate additional cost.

3) RQ_3-NON-DETERMINISM

a: BASELINE NON-DETERMINISM CHARACTERIZATION

Table 9 presents the Fleiss' Kappa and Raw Percentage Agreement scores for each model-dataset combination, without any additional strategy to mitigate potential non-determinism.

Qwen-2.5 32B achieves outstanding consistency with perfect agreement for Ansible and Puppet datasets and near-perfect performance for Chef ($k = 0.9862$). Qwen-2.5 14B maintains high reliability across all platforms ($k = 0.9713$-0.9910), while GPT-4o-mini delivers strong performance with Kappa scores above 0.89 for all datasets.

TABLE 9. SecLLM consistency analysis: Fleiss' Kappa and raw percentage agreement.

	Ansible		Chef		Puppet	
Model	Kappa	Agreemen	Kappa	Agreemen	Kappa	Agreemen
GPT-4o-mini	0.9936	0.9964	0.9066	0.9266	0.8969	0.9241
Qwen-2.5 14B	0.9779	0.9875	0.9910	0.9928	0.9713	0.9787
Qwen-2.5 32B	**1.0000**	**1.0000**	**0.9862**	**0.9890**	**1.0000**	**1.0000**

All models demonstrate a high level of consistency for most security smell types, with only "Hard-coded secret" patterns showing occasional minor variations due to their contextual complexity.

b: MITIGATION STRATEGY EVALUATION

The results for the different mitigation mechanisms for non-determinism were noteworthy.

Confidence-Based Filtering. Regarding the confidence-based filtering strategy, a comparison of accurate and inaccurate predictions is shown in Table 10, along with the ideal thresholds that maximize F1 Scores for each configuration. The reported thresholds represent aggregated performance across all nine security smells, and individual smells may require different thresholds. For example, Hard-coded Secret false positives (Table 7) cluster at high confidence (0.92-0.97), requiring threshold $\geq$0.96. The Wilcoxon Rank-Sum test indicates that confidence scores significantly discriminate between correct and incorrect predictions in eight of nine configurations. The exception is GPT-4o-mini on Ansible ($p = 0.1189$), where the minimal difference ($\Delta = 0.0023$) suggests that confidence scores are unreliable as a filtering signal. The degree of separation fluctuates considerably, though. Practical discrimination varies, as indicated by the confidence gap (Δ), which ranges from 0.0091 (GPT-4o-mini on Puppet) to 0.1194 (GPT-4o-mini on Chef). Confidence-based filtering works well for Ansible with Qwen models, where both LLM versions continue to perform effectively with a threshold between 0.80 and 0.90 (Precision: 0.9737-1.0000; Recall: 0.9459-1.0000). On the other hand, Puppet configurations achieve only moderate precision and recall, even with higher thresholds ($\approx$ 0.95). At lower thresholds (0.65–0.80), Chef achieves perfect recall but moderate precision (0.83–0.91).

These findings show that the confidence-based filtering is effective for Ansible and somewhat so for Chef, while for Puppet more techniques are needed to manage prediction uncertainty effectively.

Ensemble-Based Consensus. Table 11 compares the detection accuracy of the ensemble strategy with the results from RQ1 for all three IaC technologies and LLM models. The ensemble maintained the strong results of Qwen-2.5 32B while improving performance for GPT-4o-mini and Qwen-2.5 14B. Up to three points were gained by GPT-4o-mini, which achieved perfect precision (1.00) and F1-scores of 0.98 (Ansible), 0.92 (Chef), and 0.87 (Puppet). The larger 32B version of Qwen-2.5 remained stable at 0.89, 0.92, and 0.99 in terms of F1-score values, while Qwen-2.5 14B came in second with gains of about four points (F1-scores of 0.94, 0.91, and 0.88).

The gain comes from the ensemble's ability to smooth out individual errors through majority voting, cutting down the false positives that once hurt precision. Additionally, the ensemble eliminated the variability observed in single runs by achieving full determinism, with raw agreement and Fleiss' Kappa both equal to 1.00. Efficiency is the primary cost, as compared to RQ1; computation, inference time, and token use all increase by about five times.

Response Caching. With Fleiss' Kappa reaching 1.00 and every run agreeing flawlessly, the caching setup yielded entirely repeatable results. In other words, once caching was enabled, the models stopped oscillating between slightly different answers. Table 12 shows how detection accuracy compares with the baseline results from RQ1, and, interestingly, the accuracy did not budge. For all IaC technologies, GPT-4o-mini achieved precision and recall of about 0.98 and 0.97, respectively. The performance of Qwen-2.5 14B remains stable, with 0.91 precision and 0.92 recall, while the larger 32B version landed just a touch higher (0.93 precision, 0.94 recall).

What is also worth noting is the efficiency gain. Token usage dropped to about one-fifth of its previous level. That is mostly because prompts were repeated from memory rather than being hit against the LLM API again.

VI. RQ3-SUMMARY AND IMPLICATIONS

The results discussed above lead us to answer RQ_3 as follows:

Answer to RQ_3

SecLLM achieves high baseline consistency (K = 0.89-1.00) across all configurations. Three mitigation strategies address residual non-determinism: caching eliminates variability at 20% cost, ensemble voting guarantees reproducibility at 5× cost with 2-4% precision gains, and confidence filtering enables uncertainty quantification with IaC-specific thresholds. Optimal deployment combines caching as default with conditional ensemble or filtering based on privacy constraints and quality requirements.

Table 13 synthesizes the performance, determinism, and cost trade-offs across all mitigation configurations. Organizations should select strategies based on deployment context: *routine security audits* benefit from caching (perfect determinism at 20% cost), *compliance-critical scenarios* justify ensemble voting (2–4% precision gains at 5× cost), and *expert-validated workflows* leverage confidence filtering (efficiency maintained with manual review of uncertain predictions). To maintain data sovereignty, privacy-sensitive organizations need to deploy local models (Qwen-2.5), albeit at the expense of slight accuracy trade-offs ($\approx$2 F1 points).

These results offer several implications for practitioners and researchers:

- **Model selection impacts baseline determinism.** Qwen-2.5 32B is remarkably consistent, even without

TABLE 10. Confidence analysis: Wilcoxon test results and optimal filtering thresholds.

Model	Tech.	Correct Mean	Incorrect Mean	Δ Conf.	W Stat.	P-value	Opt. Thr.	Prec.	Recall
GPT-4o-mini	Ansible	0.9732	0.9709	0.0023	1.18	0.1189	0.75	0.9912	1.0000
	Chef	0.9635	0.8441	0.1194	4.50	**0.000003**	0.65	0.9060	1.0000
	Puppet	0.9601	0.9510	0.0091	3.63	**0.000142**	0.95	0.7909	0.8286
Qwen-2.5 14B	Ansible	0.9618	0.8813	0.0805	2.16	**0.0156**	0.90	1.0000	0.9459
	Chef	0.9673	0.9360	0.0313	4.65	**0.000002**	0.75	0.8258	1.0000
	Puppet	0.9666	0.9354	0.0312	5.35	**<0.000001**	0.95	0.7849	0.8202
Qwen-2.5 32B	Ansible	0.9814	0.9454	0.0360	2.14	**0.0161**	0.80	0.9737	1.0000
	Chef	0.9590	0.9366	0.0224	2.71	**0.0033**	0.80	0.8819	1.0000
	Puppet	0.9419	0.9182	0.0237	2.72	**0.0033**	0.80	0.8361	1.0000

TABLE 11. Ensemble-based consensus: Detection accuracy across IaC technologies.

	GPT-4o-mini Ensemble			Qwen-2.5 14B Ensemble			Qwen-2.5 32B Ensemble		
IaC Technology	Precision	Recall	F1-score	Precision	Recall	F1-score	Precision	Recall	F1-score
Ansible	**1.00**	0.97	0.98	0.95	0.94	0.94	**1.00**	**0.99**	**0.99**
Chef	**0.92**	**0.94**	**0.92**	0.91	0.92	0.91	**0.92**	0.93	**0.92**
Puppet	0.84	**0.92**	0.87	**0.89**	0.89	0.88	**0.89**	0.90	**0.89**
Average	0.92	**0.94**	0.92	0.92	0.92	0.91	**0.94**	**0.94**	**0.93**

TABLE 12. Detection accuracy with caching enabled across IaC technologies.

	GPT-4o-mini Cache			Qwen-2.5 14B Cache			Qwen-2.5 32B Cache		
IaC Technology	Prec.	Recall	F1	Prec.	Recall	F1	Prec.	Recall	F1
Ansible	**1.00**	0.97	0.98	0.94	0.94	0.94	**1.00**	**0.99**	**0.99**
Chef	0.90	**0.93**	0.91	0.91	0.92	0.91	**0.91**	**0.93**	**0.92**
Puppet	0.84	**0.92**	0.87	**0.89**	0.90	**0.89**	**0.89**	0.90	**0.89**
Average	**0.98**	**0.97**	**0.97**	0.91	0.92	0.91	0.93	0.94	0.93

TABLE 13. Combined strategy analysis: Performance, Determinism, and Cost trade-offs.

Configuration	Model	Precision	Recall	F1-score	Fleiss' K.	Comp. Cost	Privacy
Baseline	GPT-4o-mini	0.90	0.94	0.92	0.93	1.0x	Cloud
	Qwen 14B	0.92	0.92	0.91	0.98	1.0x	**Local**
	Qwen 32B	0.94	0.94	0.93	**1.00**	1.0x	**Local**
Confidence Filter	GPT-4o-mini	0.89	0.94	0.91	0.93	1.0x	Cloud
	Qwen 14B	0.87	0.92	0.89	0.98	1.0x	**Local**
	Qwen 32B	0.90	1.00	0.95	**1.00**	1.0x	**Local**
Ensemble	GPT-4o-mini	0.92	0.94	0.92	**1.00**	5.0x	Cloud
	Qwen 14B	0.92	0.92	0.91	**1.00**	5.0x	**Local**
	Qwen 32B	**0.94**	**0.94**	**0.93**	**1.00**	5.0x	**Local**
Caching	GPT-4o-mini	0.91	0.94	0.92	**1.00**	**0.2x**	Cloud
	Qwen 14B	0.91	0.92	0.91	**1.00**	**0.2x**	**Local**
	Qwen 32B	0.93	0.94	0.93	**1.00**	**0.2x**	**Local**

mitigation strategies, whereas GPT-4o-mini shows moderate variation. For security-critical applications, practitioners should weigh determinism alongside accuracy when selecting a model.

- **Confidence scores require technology-specific calibration.** While statistically significant in 8 of 9 configurations, confidence gaps vary widely ($\Delta = 0.009$–0.119). Effective filtering requires calibrated thresholds for each IaC technology and integration with human review workflows.
- **Caching provides optimal cost-determinism balance.** With 80% token reduction, perfect reproducibility, and preserved accuracy, caching emerges as the default strategy for iterative security audits and CI/CD pipelines with recurring infrastructure patterns.
- **Ensemble voting suits consistency-critical scenarios.** Perfect determinism makes ensembles appropriate for compliance audits where reproducibility is mandatory, justifying the computational overhead when regulatory requirements outweigh efficiency concerns. Future work should explore adaptive strategies that activate ensemble mode only for low-confidence predictions.
- **Privacy constraints require local model deployment.** To preserve data sovereignty, organizations handling sensitive infrastructure code must use Qwen-2.5 models, albeit with small accuracy trade-offs (2-point F1 gap).
- **Determinism must become a standard evaluation dimension.** Traditional precision–recall metrics fall short for non-deterministic systems. Evaluations of LLM-based tools should pair accuracy with measures like Fleiss' Kappa or raw agreement across runs to better capture consistency and reliability.

A. DEPLOYMENT CONSIDERATIONS AND FUTURE WORK

The empirical evaluation yields three critical insights for practitioners deploying LLM-based security-smell detection in production environments.

First, SecLLM's consistent superiority over rule-based approaches (97.8% of detection scenarios) demonstrates that semantic understanding fundamentally outperforms pattern matching for context-dependent vulnerabilities. Second, identifying specific failure modes provides organizations with clear deployment guidelines rather than opaque recommendations. Third, the economic viability ($0.003 - 0.015$ per script) and determinism mitigation strategies suggest that LLM-based detection is not merely a research prototype but a deployable solution.

Nonetheless, SecLLM requires careful analysis of hardware and operational trade-offs for production deployment.

The first consideration is related to the deployment type. Local deployment guarantees data sovereignty but at the expense of significant infrastructure; for example, Qwen-2.5 32B runs 60–120 seconds per script and needs about 96 GB of RAM. However, with about $0.003 per script and 1–5 seconds of inference time, cloud deployment (for example, through GPT-4o-mini) offers an accessible entry point.

The second consideration concerns the balance among accuracy, cost, and privacy. To this end, the framework supports three validated modes: ensemble (with high cost but maximum accuracy), caching (with low cost and deterministic), and confidence filtering (with no overhead but requires human-in-the-loop).

The final consideration is that prompt quality strongly influences performance. All this means that designing effective prompts may introduce adoption barriers, given that detecting subtle security patterns requires specialized domain expertise.

Future work should investigate several directions. One possible strategy to improve detection accuracy and broaden the list of security smells is to enhance prompt design. The use of user-in-the-loop feedback to iteratively adjust prompts and heuristics is the subject of a second research line. Finally, investigating open-weight, smaller, and more accessible LLMs for local deployments would be a viable solution to reduce operational costs.

B. THREATS TO VALIDITY

A potential threat to internal validity arises from LLMs' non-deterministic behavior, which can complicate reproducibility. The RQ3 analysis, however, mitigates this through Response Caching and Ensemble Voting, achieving perfect determinism (Fleiss' $K = 1.00$). Remaining risks are mostly deployment-related; that is, model updates can change outputs, so organizations must version configurations and validate results when upgrading. Prompt engineering remains a subtle threat to internal validity. Despite the structured approach (Persona Pattern, COSTAR, Few-shot Learning), some risks persist. Indeed, prompts are specific to the nine smells we studied and may not generalize; context limitations still allow edge-case false positives (see Section V-A1), and prompts can break with minor wording changes or model updates. To this aim, organizations extending SecLLM to new vulnerability patterns must re-engineer prompts with domain expertise and validate outputs through pilot testing. A potential construct threat is that we assume GLITCH oracle datasets reflect reliable ground truth, but this is not guaranteed. Reis et al. [31] showed that involving code authors can uncover annotation inconsistencies. Therefore, before being considered final for high-stakes security decisions, SecLLM outputs should be reviewed by humans and checked against organization-specific security policies. Regarding external validity, our study focused on Ansible, Chef, and Puppet. While widely used, these technologies may not encompass all possible configurations. Saavedra et al.'s established benchmarks [8] helped us mitigate this, but companies using other IaC technologies should perform pilot validations before production deployment.

VII. CONCLUSION

This paper introduces SecLLM, a framework that leverages LLMs to detect security smells in IaC scripts via direct semantic analysis. The empirical evaluation shows substantial gains over state-of-the-art tools across Ansible, Chef, and Puppet IaC technologies, at cost-effective prices. In order to address LLM non-determinism, SecLLM provides three mitigation strategies: response caching (perfect

reproducibility at 20% cost), ensemble voting (2-4% precision gains at 5x cost), and confidence filtering (uncertainty quantification with human-in-the-loop). These strategies and deployment options can be selected by organizations based on their review capacity, accuracy requirements, and privacy restrictions. SecLLM's prompt-based architecture enables rapid adaptation to new security smells and IaC technologies without code modifications. However, the framework's effective deployment requires security expertise for prompt engineering and validation, introducing adoption barriers. Further research could investigate new LLM architectures for security analysis, enhance prompting strategies to improve detection precision, and expand the framework's applicability to additional IaC technologies.

DATA AVAILABILITY

Their code and data are available at https://github.com/gadevito/SecLLM

REFERENCES

[1] M. Guerriero, M. Garriga, D. A. Tamburri, and F. Palomba, "Adoption, support, and challenges of Infrastructure-as-code: Insights from industry," in *Proc. IEEE Int. Conf. Softw. Maintenance Evol. (ICSME)*, Sep. 2019, pp. 580–589.

[2] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases Through Build, Test, and Deployment Automation*. London, U.K.: Pearson, 2010.

[3] M. Artac, T. Borovssak, E. Di Nitto, M. Guerriero, and D. A. Tamburri, "DevOps: Introducing infrastructure-as-code," in *Proc. IEEE/ACM 39th Int. Conf. Softw. Eng. Companion (ICSE-C)*, May 2017, pp. 497–498.

[4] K. Morris, *Infrastructure As Code*. Sebastopol, CA, USA: O'Reilly Media, 2020.

[5] A. Rahman, C. Parnin, and L. Williams, "The seven sins: Security smells in infrastructure as code scripts," in *Proc. IEEE/ACM 41st Int. Conf. Softw. Eng. (ICSE)*, May 2019, pp. 164–175.

[6] A. Rahman and L. Williams, "Different kind of smells: Security smells in infrastructure as code scripts," *IEEE Secur. Privacy*, vol. 19, no. 3, pp. 33–41, May 2021.

[7] E. Ntentos, N. E. Lueger, G. Simhandl, U. Zdun, S. Schneider, R. Scandariato, and N. E. Díaz Ferreyra, "On the understandability of design-level security practices in infrastructure-as-code scripts and deployment architectures," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 1, pp. 1–37, Jan. 2025.

[8] N. Saavedra and J. F. Ferreira, "GLITCH: Automated polyglot security smell detection in infrastructure as code," in *Proc. 37th IEEE/ACM Int. Conf. Automated Softw. Eng.* New York, NY, USA: Association for Computing Machinery, 2022, pp. 1–12, doi: 10.1145/3551349.3556945.

[9] A. Rahman, M. R. Rahman, C. Parnin, and L. Williams, "Security smells in ansible and chef scripts: A replication study," *ACM Trans. Softw. Eng. Methodology*, vol. 30, no. 1, pp. 1–31, Jan. 2021.

[10] A. Rahman and C. Parnin, "Detecting and characterizing propagation of security weaknesses in puppet-based infrastructure management," *IEEE Trans. Softw. Eng.*, vol. 49, no. 6, pp. 3536–3553, Jun. 2023.

[11] S. Sarsa, P. Denny, A. Hellas, and J. Leinonen, "Automatic generation of programming exercises and code explanations using large language models," in *Proc. ACM Conf. Int. Comput. Educ. Res.*, 2022, pp. 27–43.

[12] D. Sobania, M. Briesch, C. Hanna, and J. Petke, "An analysis of the automatic bug fixing performance of ChatGPT," 2023, *arXiv:2301.08653*.

[13] A. Ahmad, M. Waseem, P. Liang, M. Fehmideh, M. S. Aktar, and T. Mikkonen, "Towards human-bot collaborative software architecting with ChatGPT," 2023, *arXiv:2302.14600*.

[14] S. Ouyang, J. M. Zhang, M. Harman, and M. Wang, "An empirical study of the non-determinism of ChatGPT in code generation," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 2, pp. 1–28, Feb. 2025.

[15] M. Astekin, M. Hort, and L. Moonen, "An exploratory study on how non-determinism in large language models affects log parsing," in *Proc. ACM/IEEE 2nd Int. Workshop Interpretability, Robustness, Benchmarking Neural Softw. Eng.*, Apr. 2024, pp. 13–18.

[16] S. Dalla Palma, D. Di Nucci, F. Palomba, and D. A. Tamburri, "Toward a catalog of software quality metrics for infrastructure code," *J. Syst. Softw.*, vol. 170, Dec. 2020, Art. no. 110726.

[17] R. Opdebeeck, A. Zerouali, and C. De Roover, "Smelly variables in ansible infrastructure code: Detection, prevalence, and lifetime," in *Proc. IEEE/ACM 19th Int. Conf. Mining Softw. Repositories (MSR)*, May 2022, pp. 61–72.

[18] *CWE-Common Weakness Enumeration*. Accessed: Nov. 22, 2024. [Online]. Available: https://cwe.mitre.org/index.html

[19] A. Rahman, E. Farhana, C. Parnin, and L. Williams, "Gang of eight: A defect taxonomy for infrastructure as code scripts," in *Proc. IEEE/ACM 42nd Int. Conf. Softw. Eng. (ICSE)*, Oct. 2020, pp. 752–764.

[20] A. Rahman, E. Farhana, and L. Williams, "The 'as code' activities: Development anti-patterns for infrastructure as code," *Empirical Softw. Eng.*, vol. 25, no. 5, pp. 3430–3467, Sep. 2020.

[21] M. Chen et al., "Evaluating large language models trained on code," 2021, *arXiv:2107.03374*.

[22] E. Kasneci et al., "ChatGPT for good? On opportunities and challenges of large language models for education," *Learn. Individual Differences*, vol. 103, Jan. 2023, Art. no. 102274.

[23] W. Xin Zhao et al., "A survey of large language models," 2023, *arXiv:2303.18223*.

[24] M. M. Hassan, J. Salvador, S. K. K. Santu, and A. Rahman, "State reconciliation defects in infrastructure as code," *Proc. ACM Softw. Eng.*, vol. 1, no. FSE, pp. 1865–1888, Jul. 2024.

[25] J. Achiam et al., "GPT-4 technical report," 2023, *arXiv:2303.08774*.

[26] *Claude 3.5 Sonnet*. Accessed: Nov. 15, 2024. [Online]. Available: https://claude.ai/

[27] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Roziére, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, "LLaMA: Open and efficient foundation language models," 2023, *arXiv:2302.13971*.

[28] T. Sharma, M. Fragkoulis, and D. Spinellis, "Does your configuration code smell?" in *Proc. IEEE/ACM 13th Work. Conf. Mining Softw. Repositories (MSR)*, May 2016, pp. 189–200.

[29] J. Schwarz, A. Steffens, and H. Lichter, "Code smells in infrastructure as code," in *Proc. 11th Int. Conf. Quality Inf. Commun. Technol. (QUATIC)*, Sep. 2018, pp. 220–228.

[30] R. Opdebeeck, A. Zerouali, and C. De Roover, "Control and data flow in security smell detection for infrastructure as code: Is it worth the effort?" in *Proc. IEEE/ACM 20th Int. Conf. Mining Softw. Repositories (MSR)*, May 2023, pp. 534–545.

[31] S. Reis, R. Abreu, M. d'Amorim, and D. Fortunato, "Leveraging practitioners' feedback to improve a security linter," in *Proc. 37th IEEE/ACM Int. Conf. Automated Softw. Eng.*, Oct. 2022, pp. 1–12.

[32] J. White, Q. Fu, S. Hays, M. Sandborn, C. Olea, H. Gilbert, A. Elnashar, J. Spencer-Smith, and D. C. Schmidt, "A prompt pattern catalog to enhance prompt engineering with ChatGPT," 2023, *arXiv:2302.11382*.

[33] *GovTech Data Science and Singapore Government AI Division*, Prompt Engineering Playbook, Singapore, 2023.

[34] L. Reynolds and K. McDonell, "Prompt programming for large language models: Beyond the few-shot paradigm," in *Proc. CHI Conf. Human Factors Comput. Syst.*, May 2021, pp. 1–7.

[35] G. De Vito, F. Palomba, and F. Ferrucci. *Secllm Github Repository*. Accessed: Nov. 2025. [Online]. Available: https://github.com/gadevito/SecLLM

[36] Y. Ding, Z. Wang, W. U. Ahmad, H. Ding, M. Tan, N. Jain, M. K. Ramanathan, R. Nallapati, P. Bhatia, D. Roth, and B. Xiang, "CrossCodeEval: A diverse and multilingual benchmark for cross-file code completion," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 46701–46723.

[37] G. Orlanski, K. Xiao, X. García, J. Hui, J. J. Howland, J. Malmaud, J. Austin, R. Singh, and M. Catasta, "Measuring the impact of programming language distribution," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 26619–26645.

[38] B. Guan, Y. Wan, Z. Bi, Z. Wang, H. Zhang, P. Zhou, and L. Sun, "CodeIP: A grammar-guided multi-bit watermark for large language models of code," in *Proc. Findings Assoc. Comput. Linguistics, EMNLP*, 2024, pp. 9243–9258.

[39] A. Yang et al., "Qwen2.5 technical report," 2024, *arXiv:2412.15115*.

[40] H. Chang, J. Park, S. Ye, S. Yang, Y. Seo, D.-S. Chang, and M. Seo, "How do large language models acquire factual knowledge during pretraining?" in *Proc. 38th Annu. Conf. Neural Inf. Process. Syst.*, 2024, pp. 60626–60668.

[41] T. Inaba, G. Kamoda, K. Inui, M. Isonuma, Y. Miyao, Y. Oseki, B. Heinzerling, and Y. Takagi, "How LLMs learn: Tracing internal representations with sparse autoencoders," 2025, *arXiv:2503.06394*.
[42] A. Al-Kaswan, M. Izadi, and A. van Deursen, "Traces of memorisation in large language models for code," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng.*, Apr. 2024, pp. 1–12.
[43] D. D. Leybzon and C. Kervadec, "Learning, forgetting, remembering: Insights from tracking LLM memorization during training," in *Proc. 7th BlackboxNLP Workshop, Analyzing Interpreting Neural Netw. NLP*, 2024, pp. 43–57.
[44] F. Zampetti, R. Kapur, M. Di Penta, and S. Panichella, "An empirical characterization of software bugs in open-source cyber–physical systems," *J. Syst. Softw.*, vol. 192, Oct. 2022, Art. no. 111425.
[45] J. L. Fleiss, B. Levin, and M. C. Paik, *Statistical Methods for Rates and Proportions*. Hoboken, NJ, USA: Wiley, 2013.
[46] J. R. Landis and G. G. Koch, "The measurement of observer agreement for categorical data," *Biometrics*, vol. 33, no. 1, p. 159, Mar. 1977.

GABRIELE DE VITO (Member, IEEE) received the master's degree in computer science from the University of Salerno, where he is currently pursuing the Ph.D. degree. He was a CTO with various Italian IT firms. He has been a High School Teacher and a Professional Consultant, since 2021. His certifications include COSMIC and Function Point Specialist. He served on program committees for international conferences (e.g., EASE) and reviews for journals in software engineering (e.g., IEEE Transactions on Software Engineering). He has co-chaired Mensura, from 2022 to 2023. His primary research interests include software engineering, machine learning, and large language models.

FABIO PALOMBA (Member, IEEE) is currently an Associate Professor of computer science with the University of Salerno and a member of the Software Engineering (SeSa) Laboratory. His research interests include software maintenance and evaluation, empirical software engineering, and source code quality. He has been a recipient of various Distinguished Paper and Best Paper Awards. He serves on program committees for major conferences and as a referee for key journals in software engineering. He has been the Program Co-Chair of ICPC 2021 and SANER 2024, and has chaired various tracks and workshops. He serves on editorial boards for several software engineering journals, earning 14 Distinguished/Outstanding Reviewer Awards.

FILOMENA FERRUCCI is currently a Full Professor of computer science with the University of Salerno, Italy. She is the Co-Director of the SQM/Web Engineering Laboratory and a member of the SeSa Laboratory. She has co-authored more than 200 peer-reviewed publications, co-edited three books, and guest-edited two special issues. Her research interests include empirical software engineering, software metrics, machine learning, and search-based software engineering. She coordinates the Ph.D. Program in Computer Science, serves as the Program Co-Chair of the International School on Software Engineering. She is a member of the Editorial Board of IEEE Transactions on Software Engineering.